

Programação Avançada

Unidade 3: Técnicas de Programação

Aula 13: Introdução ao Paralelismo e à Programação Funcional

Prof. Eduardo Corrêa Gonçalves

17/06/2026

Sumário

Introdução

O que é programação paralela?

Por que a programação paralela é importante?

Base de Dados de Exemplo

Programação Paralela em Python

Função `map()`

Módulo `multiprocessing`

Programação Funcional em Python: `map()`, `filter()` e `reduce()`

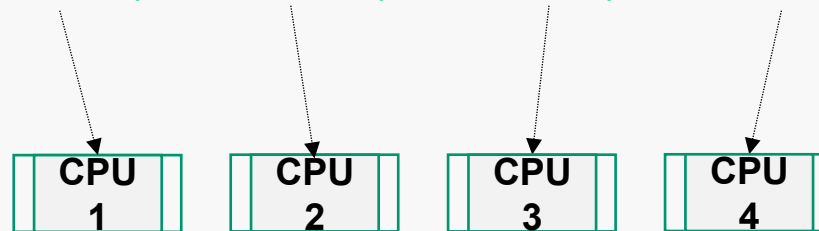
Introdução (1/2)

O que é Programação Paralela?

Forma de computação que possibilita com que **múltiplas cópias** de uma mesma funcionalidade sejam executadas ao mesmo tempo, porém processando **dados diferentes**.

Assim, **reduz-se o tempo** necessário para executar um programa.

dados = [Cliente 1, Cliente 2, Cliente 3, Cliente 4, ...]



Introdução (2/2)

Por que a Programação Paralela é importante?

Devido a **quatro** fatos/tendências que ocorrem no mundo atual:

Disponibilidade de grandes massas de dados (**big data**).

Necessidade de **adaptar os algoritmos** antigos para a realidade do big data.

Popularização das **novas tecnologias** que facilitam a programação paralela (ex.: CPU's multicore, bibliotecas e APIs, etc.).

Investimentos bilionários de **grandes empresas**: NVIDIA, Google, IBM, ...

Base de Dados de Exemplo (1/3)

Nesta aula, desenvolveremos um programa paralelo em Python.

Ele será utilizado para processar o BD de **Personalidades** abaixo:

Personalidades

nome	prof	anoNasc
Mário Quintana	Poeta	1906
Jane Austen	Escritora	1775
Federico Fellini	Cineasta	1920
Ada Lovelace	Matemática	1815
Noam Chomsky	Linguista	1928

* Os exemplos da aula foram baseados nos apresentados em Bader (2017)

Base de Dados de Exemplo (2/3)

Representação do BD em Python

```
# 1.Representando o BD de Personalidades em Python
import collections

Personalidade = collections.namedtuple(
    'Personalidade', ['nome', 'prof', 'anoNasc'])

personalidades = (
    Personalidade(nome='Mario Quintana', prof='Poeta', anoNasc=1906),
    Personalidade(nome='Jane Austen', prof='Escritora', anoNasc=1775),
    Personalidade(nome='Federico Fellini', prof='Cineasta', anoNasc=1920),
    Personalidade(nome='Ada Lovelace', prof='Matemática', anoNasc=1815),
    Personalidade(nome='Noam Chomsky', prof='Linguista', anoNasc=1928)
)
```

Base de Dados de Exemplo (3/3)

Utilizamos a ED “namedTuple” – similar ao dicionário, porém imutável

```
print(personalidades[4].nome) #Noam Chomsky
print(personalidades[4].anoNasc) #1928
print(personalidades[4].prof) #Linguista

personalidades[4].prof = 'Filósofo' #ERRO! A namedTuple é imutável

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: can't set attribute
```

Programação Paralela em Python (1/8)

O Ecossistema Python oferece muitos recursos para programação paralela.

Exemplos:

‘mpi4py’ (implementação do modelo MPI)

‘Cupy’ (versão da NumPy/SciPy para GPUs)

...

Nessa aula, apresentaremos dois recursos que, quando combinados, se destacam por sua simplicidade e popularidade:

Função `map()`:

No contexto da programação paralela, é utilizada para gerar um novo BD a partir de transformações no BD original.

Módulo `multiprocessing`:

Oferece diversas funções para simplificar o processamento paralelo.

Programação Paralela em Python (2/8)

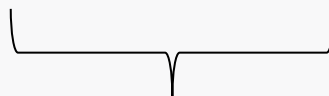
Função map()

Possibilita a geração de um **novo BD** a partir de transformações no BD original.

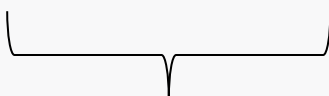
Basta especificar uma função de transformação e o BD original.

A função de transformação será automaticamente aplicada sobre todos os elementos do BD original (ela faz um loop automático).

```
novo_bd = map(f_calcula_idade, personalidades)
```



função de
transformação



BD
original

Programação Paralela em Python (3/8)

Função map():

```
#2. gera um novo BD com a idade e nome de cada personalidade
def f_calcula_idade(pessoa):
    return {"nome": pessoa.nome, "idade": 2026 - pessoa.anoNasc}

novo_bd = tuple(map(f_calcula_idade, personalidades))

print(novo_bd)
```

[Saída]:

```
({'idade': 120, 'nome': 'Mario Quintana'},
 {'idade': 251, 'nome': 'Jane Austen'},
 {'idade': 106, 'nome': 'Federico Fellini'},
 {'idade': 211, 'nome': 'Ada Lovelace'},
 {'idade': 98, 'nome': 'Noam Chomsky'})
```

Programação Paralela em Python (4/8)

Módulo multiprocessing

No exemplo anterior, a função **map()** foi executada de maneira **sequencial** em um único núcleo da CPU.

Calculou a idade do Mário Quintana, depois da Jane Austen, depois do Fellini, ...

Porém, o cálculo da idade de uma pessoa do BD **não depende** do cálculo da idade das outras.

Então por que não **paralelizar** o cálculo???

nome	prof	anoNasc	Personalidades
Mário Quintana	Poeta	1906	
Jane Austen	Escritora	1775	
Federico Fellini	Cineasta	1920	
Ada Lovelace	Matemática	1815	
Noam Chomsky	Linguista	1928	

Programação Paralela em Python (5/8)

Módulo multiprocessing

Felizmente, paralelizar esse programa em Python é simples!

Basta utilizar o módulo **multiprocessing**, que oferece diversos recursos para simplificar o processamento paralelo

Um deles é a **classe Pool**.

Ela nos permite configurar um grupo (*pool*) de processos de trabalho para serem executados em paralelo nos vários núcleos da CPU.

Com isso, torna-se possível “fazer um **map()**” em paralelo.

Programação Paralela em Python (6/8)

AVALIAÇÃO – **map()** sequencial *versus* **map()** paralelo

Código de teste: gera BD com as idades e nomes das personalidades.

```
#3. código de teste
import time
import os

def f_transformar(x):
    #imprime o id do processo e o nome da personalidade sendo processada
    print(f'processo {os.getpid()} - processando o registro {x.nome}')

    #espera 1 segundo (é um recurso para o nosso teste)
    time.sleep(1)
    print(f'FIM do processamento do registro {x.nome}')

    return {"nome": x.nome, "idade": 2026 - x.anoNasc}
```

Programação Paralela em Python (7/8)

TESTE 1 – tempo de execução do map() sequencial

```
#TESTE 1: tempo de execução do map() sequencial
inicio = time.time() #início da execução
bd1 = tuple(map(f_transformar, personalidades))
fim = time.time() #fim da execução

print(f'tempo de execucao: {fim - inicio:.2f}')
```

Programação Paralela em Python (7/8)

TESTE 1 – tempo de execução do map() sequencial

Resultado:

5.01s de execução

Apenas 1 processo criado

*processo é uma instância
do programa em execução

processo 58 - processando o registro Mario Quintana

FIM do processamento do registro Mario Quintana

processo 58 - processando o registro Jane Austen

FIM do processamento do registro Jane Austen

processo 58 - processando o registro Federico Fellini

FIM do processamento do registro Federico Fellini

processo 58 - processando o registro Ada Lovelace

FIM do processamento do registro Ada Lovelace

processo 58 - processando o registro Noam Chomsky

FIM do processamento do registro Noam Chomsky

tempo de execucao: 5.01

Programação Paralela em Python (8/8)

TESTE 2 – tempo de execução do map() paralelo

Utilizamos o módulo multiprocessing

Configuramos 5 processos que são executados em paralelo, atuando sobre registros diferentes do BD.

```
#TESTE 2: tempo de execução do map() paralelo
import multiprocessing

""" o if __name__ == "__main__": é obrigatório no multiprocessing
para evitar a criação de um loop infinito de processos """

if __name__ == '__main__':

    inicio = time.time() #início da execução

    #cria o pool de multiprocessamento
    with multiprocessing.Pool(processes=5) as pool:
        #executa pool.map()
        bd2 = tuple(pool.map(f_transformar, personalidades))

    fim = time.time() #fim da execução

    print(f'tempo de execucao: {fim - inicio:.2f}')
```


Programação Paralela em Python (8/8)

TESTE 2 – tempo de execução do map() paralelo

Resultado:

1.14s de execução

5 processos criados

processo 905 - processando o registro Mario Quintana

processo 907 - processando o registro Federico Fellini

processo 906 - processando o registro Noam Chomsky

processo 909 - processando o registro Jane Austen

processo 908 - processando o registro Ada Lovelace

FIM do processamento do registro Mario Quintana

FIM do processamento do registro Federico Fellini

FIM do processamento do registro Noam Chomsky

FIM do processamento do registro Jane Austen

FIM do processamento do registro Ada Lovelace

tempo de execucao: 1.14

Resumo – Programação Paralela

A programação paralela permite com que **múltiplas cópias** de uma mesma funcionalidade sejam executadas ao mesmo tempo, porém processando **dados diferentes**.

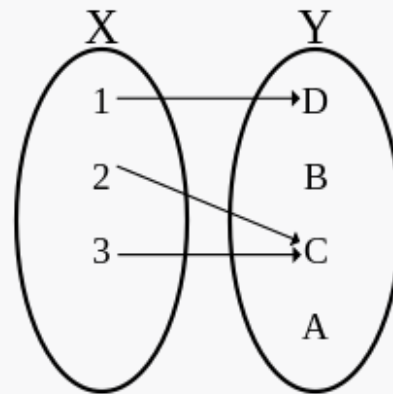
Trata-se de uma tecnologia fundamental para o processamento de Big Data.

Python oferece muitos recursos para programação paralela, sendo o uso da **função `map()`** do **módulo `multiprocessing`** um dos mais populares e de simples utilização.

As referências no último slide podem ser consultadas por aqueles que desejam conhecer mais sobre o assunto.

Programação Funcional - Introdução (1/4)

- O que é Programação Funcional (*functional programming - FP*)?
 - Como o nome indica, é um paradigma de programação em que as principais “atrizes” são as **funções**.
 - * **Importante** – na FP, o conceito de função é o mesmo de **função matemática**:
 - “relação binária sobre dois conjuntos que associa a cada elemento do primeiro conjunto exatamente um elemento do segundo conjunto”



Programação Funcional - Introdução (2/4)

- **Ausência de Efeitos Colaterais**

- A FP é baseada em uma simples premissa: as funções devem ser **puras**, não devendo produzir **efeitos colaterais** (*side effects*).
 - Uma função não deve depender de dados externos
 - *(isto é, que não tenham sido passados para função)*
 - Não pode alterar dados externos.
 - Também não pode alterar nenhum dado de entrada.
 - E não deverá fazer nada diferente de mapear um elemento do conjunto de entrada a um único elemento do conjunto de saída
 - *(caso contrário, não poderia ser definida como uma função matemática)*

* Na prática, não é possível eliminar por completo os efeitos colaterais. Sem eles, os programas fariam apenas cálculos e não poderiam gravar em bancos de dados ou integrar com sistemas externos.

Programação Funcional - Introdução (3/4)

- **Por que outro paradigma? (1/2)**
 - Neste curso, temos trabalhado com os dois paradigmas de programação mais populares:
 - **Programação estruturada**
 - Ótima para o aprendizado de programação
 - Prática para a criação de scripts de ciência de dados (pequenos scripts para a realização de experimentos)
 - **POO**
 - Mais adequada para a criação de apps, sistemas, pacotes, processos complexos de ciência de dados, bibliotecas reusáveis, etc. (programas mais complexos).
 - E por que a **programação funcional** também????

Programação Funcional - Introdução (4/4)

- Por que outro paradigma? (2/2)
 - A **FP** não é tão popular, mas também é importante porque:
 - Seu modelo **facilita a execução paralela** de processos.
 - Ela Introduziu conceitos que acabaram sendo adotados por diferentes ferramentas importantes da ciência de dados. Por exemplo:
 - **map** e **reduce** (principais primitivas da FP) são os elementos centrais do **Hadoop**, tecnologia voltada para o processamento paralelo de Big Data.
 - A primitiva **map** é comumente utilizada no **R** para transformar dados.
 - A **pandas** adota (ainda que de forma embutida) conceitos de programação funcional

Programação Funcional em Python (1/11)

Agora, apresentaremos uma breve introdução à FP em Python.

- Principalmente, como trabalhar com as principais primitivas da FP: **filter**, **map** e **reduce**.
- **filter()**
 - Possibilita a geração de uma nova ED contendo apenas alguns elementos selecionados (**filtrados**) da ED original.
 - Exemplo – obter todas as personalidades nascidas antes do Século XX:

```
#filter
pers_antes_20 = filter(lambda x: x.anoNasc < 1901, personalidades)

for pessoa in pers_antes_20: print(pessoa)
```

>

Personalidade(nome='Jane Austen', prof='Escritora', anoNasc=1775)

Personalidade(nome='Ada Lovelace', prof='Matemática', anoNasc=1815)

Programação Funcional em Python (2/11)

- **filter()** - A função recebe 2 parâmetros:
 - função de filtragem (pode ser lambda ou uma função comum).
 - Coleção de dados.

```
filter(lambda x: x.anoNasc < 1901, personalidades)
```

função de filtragem

coleção

- A função de filtragem deve ser booleana. Ela será automaticamente aplicada a cada elemento x da coleção.
 - Não é preciso programar um laço, pois `filter()` faz isso sozinho.
- Como resultado, será retornado um **iterador** de onde poderemos extrair os elementos filtrados.
 - *isso pode ser feito com um for, como no exemplo anterior...*
 - *... ou convertendo o iterador para uma lista ou tupla*

Programação Funcional em Python (3/11)

- Exemplo2 – obter todas as personalidades nascidas antes do Século XX que são escritoras.
- Nesse exemplo, usamos `tuple()` para converter o iterador retornado em uma tupla.

```
#filter - exemplo 2 usando lambda
resultado = filter(lambda x: x.anoNasc < 1901 and x.prof == 'Escritora',
                  personalidades)

print(tuple(resultado))
```

>

(Personalidade(nome='Jane Austen', prof='Escritora', anoNasc=1775),)

Programação Funcional em Python (4/11)

- Exemplo3 – Podemos usar funções comuns e não apenas lambdas
 - isso vale para o filter(), map() e reduce()

```
def f_antes_seculo_xx(x):
    return x.anoNasc < 1901

resultado = filter(f_antes_seculo_xx, personalidades)
print(tuple(resultado))
```

>

```
(Personalidade(nome='Jane Austen', prof='Escritora', anoNasc=1775),
 Personalidade(nome='Ada Lovelace', prof='Matemática', anoNasc=1815))
```

Programação Funcional em Python (5/11)

- `map()` – Como já vimos, possibilita a geração de uma nova ED a partir da **transformações** nos dados da ED original.
- Exemplo – obter o nome e idade de cada cientista

```
map(lambda x: {"nome": x.nome, "idade": 2026 - x.anoNasc}, personalidades)
```



função de filtragem

coleção

- Será retornado um **iterador** de onde poderemos extrair os elementos transformados.
 - Ele será o resultado da aplicação da função de transformação a todos os elementos da coleção.

Programação Funcional em Python (6/11)

- `map()` – exemplo 2
- No exemplo abaixo, gera BD com o nome convertido para maiúsculo e a idade

```
#map - exemplo 2 -> converte o nome para maiúsculo
bd2 = map(lambda x: {"nome": x.nome.upper(), "idade": 2026 - x.anoNasc},
           personalidades)

print(tuple(bd2))
```

>

```
({'nome': 'MARIO QUINTANA', 'idade': 120}, {'nome': 'JANE AUSTEN', 'idade': 251}, {'nome': 'FEDERICO FELLINI', 'idade': 106}, {'nome': 'ADA LOVELACE', 'idade': 211}, {'nome': 'NOAM CHOMSKY', 'idade': 98})
```

Programação Funcional em Python (7/11)

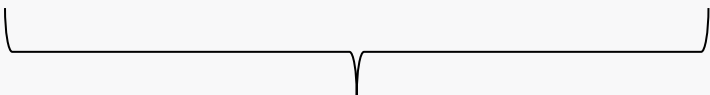
- `reduce()`

Recebe uma função e uma coleção de itens como entrada.

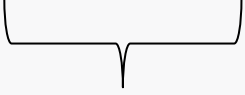
Retorna um **resultado agregado**, computado sobre os itens

Exemplo – obter a soma das idades das personalidades

`reduce(lambda a, x: a + (2026 - x.anoNasc), personalidades, 0)`



função que computa o
agregado



coleção



valor inicial do
acumulador

Programação Funcional em Python (8/11)

- `reduce()`
 - Veja que dessa vez a **função lambda** recebe dois argumentos: *a* e *x*
 - *a* (**acumulador**): valor retornado pela execução da função lambda no **elemento anterior** da coleção.
 - *x* (**valor**): valor do elemento da coleção correntemente processado

```
reduce(lambda a, x: a + (2026 - x.anoNasc), personalidades, 0)
```

- Em nossos exemplo, a cada iteração, o valor do acumulador *a* é atualizado pela expressão **$a + (2026 - x.anoNasc)$** .

Programação Funcional em Python (9/11)

- `reduce()`
 - Já a função **`reduce()`** propriamente dita possui três parâmetros:

```
reduce(lambda a, x: a + (2026 - x.anoNasc), personalidades, 0)
```

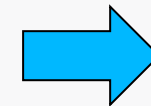
- A função de redução. Em nosso caso é uma lambda, mas pode ser uma função comum
- A coleção de entrada (personalidades)
- 0 – este é o valor inicial do acumulador *a*. Como em nosso exemplo, estamos gerando uma soma, o valor inicial foi definido como 0.

Programação Funcional em Python (10/11)

- **reduce()**
 - Significado do reduce abaixo:
 - “Reduza a coleção personalidades a um valor único, usando a regra definida na função lambda, iniciando com o valor 0 para o acumulador”

reduce(lambda a, x: a + (2026 - x.anoNasc), personalidades, 0)

nome	prof	anoNasc
Mário Quintana	Poeta	1906
Jane Austen	Escritora	1775
Federico Fellini	Cineasta	1920
Ada Lovelace	Matemática	1815
Noam Chomsky	Linguista	1928



786

Programação Funcional em Python (11/11)

- `reduce()` – exemplo completo
- Ao contrário de `filter()` e `map()`, para utilizar `reduce()` precisamos importar o módulo **functools**.

```
#reduce
from functools import reduce

soma = reduce(lambda a, x: a + (2026 - x.anoNasc), personalidades, 0)

print(soma)
```

> 786

Resumo – Programação Funcional

É um **outro paradigma de programação**, diferente da programação estruturada e da POO.

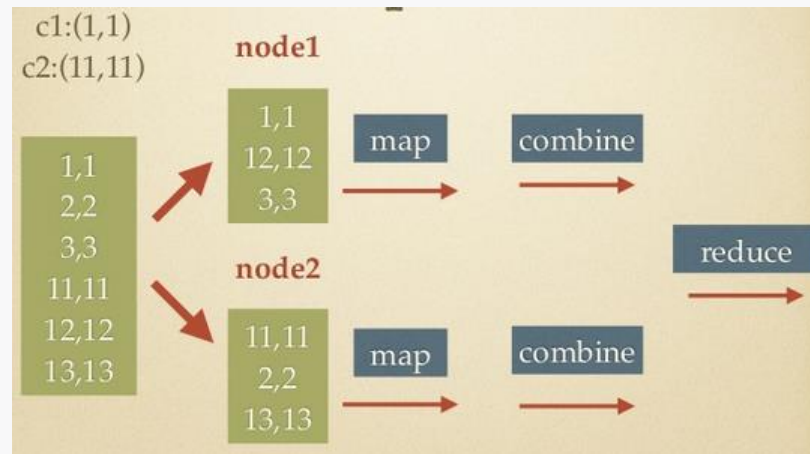
Oferece uma **visão matemática** da programação, onde todas as ações devem ser executadas por **funções puras**.

Principal **característica positiva**: um programa funcional é, em geral, mais **fácil** de ser **paralelizado**

De fato, o modelo MapReduce tornou-se o elemento central do Hadoop, tecnologia voltada para o processamento paralelo de Big Data.

Resumo – Programação Funcional

- Vários algoritmos podem ser adaptados para o modelo MapReduce tornando facilitada a sua execução em paralelo.
 - Alguns de forma bem simples, outros com mais modificações.
- Para citar um exemplo, o k-médias* é facilmente adaptável.



*Zhao W., Ma, H. e He, Q. (2009). "Parallel K-Means Clustering Based on MapReduce", IEEE CloudCom.

Referências

Langford, T. (2020). “Parallel Programming with Python”.

<https://www.youtube.com/watch?v=AG1soUh4-nU>

- Bader, D. (2017). “Functional Programming in Python”.

https://www.youtube.com/watch?v=xJCPpDIk9_w&list=PLP8GkvalxJP1z5bu4NX_bFrEInBkAgTMr&index=1

Hunt, J. (2019). “Advanced Guide to Python 3 Programming”. Springer, (Cap. 29)